

Cheat Sheet: Exploring Dart Language

Variables and types

Type	Description	Code example
var	Inferred type variable	   <pre>var name = 'Dart';</pre>
Dynamic	Variable types can change dynamically	   <pre>dynamic x = 42;</pre>
Final	Constant at runtime	   <pre>final cityName = 'New York';</pre>
Const	Compile-time constant	   <pre>const PI = 3.14;</pre>

Functions and methods

Type	Description	Code example
Function	Defines a function that adds two numbers	 dart  <pre>int add(int a, int b) => a + b;</pre>
Arrow syntax	Uses arrow syntax for concise function declaration	 dart  <pre>void printItem(item) => print(item);</pre>
Required parameters	Must be explicitly provided in the function call	 dart  <pre>int multiply(int a, int b) => a * b;</pre>
Optional positional parameters	Can be omitted; enclosed in square brackets	 dart  <pre>String fullName(String firstName, [String? middleName, String? lastName])</pre>
Named parameters	Specified by name; can be required or optional with default values, enclosed in curly braces	 dart  <pre>void greet({required String name, String greeting = 'Hello'}) => print('\$greeting, \$name!');</pre>

Default parameters Allows default values if not provided in the function call



dart



```
String describe(String name, {int age = 30, String city = 'Unknown'}) =>
    '$name is $age years old and lives in $city';
```

Closures Anonymous functions that can capture variables from their context



dart



```
List<int> numbers = [1, 2, 3];
numbers.forEach((number) => print(number * 2));
```

Classes and OOP

Type	Description	Code example
Class	Defines a simple class with properties	 dart  <pre> class Person { String name; int age; Person(this.name, this.age); } </pre>
Inheritance	Demonstrates basic inheritance in Dart	 dart  <pre> class Employee extends Person { int salary; Employee(String name, int age, this.salary) : super(name, age); } </pre>
Encapsulation	Uses private variables (underscore) and methods to restrict access	 dart  <pre> class Person { String name; // Public property int _age; // Private property, underscore prefix in Dart } </pre>
Public properties	Can be accessed from any location where the object is visible	 dart  <pre> class Person { String name; // Public property } </pre>
Private properties	Prefixed with an underscore and can only be accessed within the same library (file)	 dart  <pre> class Person { int _age; // Private property } </pre>

Getters and Setters

Control access to class properties



dart



```
class Person {  
  int _age = 0;  
  int get age => _age; // Getter  
  set age(int value) { // Setter  
    _age = value;  
  }  
}
```

Static methods

Belong to the class rather than any instance of the class and can be called without an object



dart



```
class Utility {  
  static int add(int a, int b) {  
    return a + b;  
  }  
}
```

Anonymous functions

Used for single-expression functions; also known as lambdas or closures



dart



```
var list = ['apples', 'bananas', 'oranges'];  
list.forEach((item) {  
  print(item);  
});
```

Common data structures

Type	Description	Code example
List	Ordered collection of items	 dart  <pre>List<int> numbers = [1, 2, 3];</pre>
Map	Key-value pairs collection	 dart  <pre>Map<String, int> ages = {'Alice': 18, 'Bob': 20};</pre>
Set	Unordered collection of unique items	 dart  <pre>Set<String> names = {'Alice', 'Bob'};</pre>
Queues	FIFO collection for elements	 dart  <pre>import 'dart:collection'; Queue<int> queue = Queue(); queue.addAll([1, 2, 3]);</pre>
LinkedLists	Sequence of elements where each element points to the next	 dart  <pre>import 'dart:collection'; class Item extends LinkedListEntry<Item> { int value; Item(this.value); } LinkedList<Item> list = LinkedList<Item>(); list.add(Item(1));</pre>

Libraries and command line utilities

Type	Description	Code example
import	Access built-in Dart libraries	 dart  <pre>import 'dart:math';</pre>
CLI commands	Compile Dart code to native executable	 dart  <pre>dart compile exe test.dart</pre>
Dart SDK	Essential tool for running and managing Dart applications	 dart  <pre>dart run, dart create</pre>
Pub tool	Package manager to handle dependencies	 dart  <pre>dart pub get, dart pub add http</pre>
Dart DevTools	Suite for debugging and performance profiling	 dart  <pre>Performance profiling, memory analysis, and widget inspection</pre>
dart:core	Fundamental classes and functions	 dart  <pre>Handling strings, numbers, collections like List and Map.</pre>
dart:math	Provides mathematical constants and functions	 dart  <pre>sin, cos, sqrt, and constants like pi.</pre>

dart:async Supports asynchronous programming



dart



Future, Stream **for** handling asynchronous operations.

dart:convert Handling JSON, UTF-8 encoding/decoding



dart



jsonEncode, jsonDecode **for** JSON data manipulation.

Custom libraries Creating and using your own library



dart



```
library my_utils; int add(int a, int b) => a + b;
```

Cheat Sheet: Exploring Dart Language

Variables and types

Type	Description	Code example
var	Inferred type variable	   <pre>var name = 'Dart';</pre>
Dynamic	Variable types can change dynamically	   <pre>dynamic x = 42;</pre>
Final	Constant at runtime	   <pre>final cityName = 'New York';</pre>
Const	Compile-time constant	   <pre>const PI = 3.14;</pre>

Functions and methods

Type	Description	Code example
Function	Defines a function that adds two numbers	 dart  <pre>int add(int a, int b) => a + b;</pre>
Arrow syntax	Uses arrow syntax for concise function declaration	 dart  <pre>void printItem(item) => print(item);</pre>
Required parameters	Must be explicitly provided in the function call	 dart  <pre>int multiply(int a, int b) => a * b;</pre>
Optional positional parameters	Can be omitted; enclosed in square brackets	 dart  <pre>String fullName(String firstName, [String? middleName, String? lastName])</pre>
Named parameters	Specified by name; can be required or optional with default values, enclosed in curly braces	 dart  <pre>void greet({required String name, String greeting = 'Hello'}) => print('\$greeting, \$name!');</pre>

Default parameters Allows default values if not provided in the function call



dart



```
String describe(String name, {int age = 30, String city = 'Unknown'}) =>
    '$name is $age years old and lives in $city';
```

Closures Anonymous functions that can capture variables from their context



dart



```
List<int> numbers = [1, 2, 3];
numbers.forEach((number) => print(number * 2));
```

Classes and OOP

Type	Description	Code example
Class	Defines a simple class with properties	 dart  <pre> class Person { String name; int age; Person(this.name, this.age); } </pre>
Inheritance	Demonstrates basic inheritance in Dart	 dart  <pre> class Employee extends Person { int salary; Employee(String name, int age, this.salary) : super(name, age); } </pre>
Encapsulation	Uses private variables (underscore) and methods to restrict access	 dart  <pre> class Person { String name; // Public property int _age; // Private property, underscore prefix in Dart } </pre>
Public properties	Can be accessed from any location where the object is visible	 dart  <pre> class Person { String name; // Public property } </pre>
Private properties	Prefixed with an underscore and can only be accessed within the same library (file)	 dart  <pre> class Person { int _age; // Private property } </pre>

Getters and Setters

Control access to class properties



dart



```
class Person {  
  int _age = 0;  
  int get age => _age; // Getter  
  set age(int value) { // Setter  
    _age = value;  
  }  
}
```

Static methods

Belong to the class rather than any instance of the class and can be called without an object



dart



```
class Utility {  
  static int add(int a, int b) {  
    return a + b;  
  }  
}
```

Anonymous functions

Used for single-expression functions; also known as lambdas or closures



dart



```
var list = ['apples', 'bananas', 'oranges'];  
list.forEach((item) {  
  print(item);  
});
```

Common data structures

Type	Description	Code example
List	Ordered collection of items	 dart  <pre>List<int> numbers = [1, 2, 3];</pre>
Map	Key-value pairs collection	 dart  <pre>Map<String, int> ages = {'Alice': 18, 'Bob': 20};</pre>
Set	Unordered collection of unique items	 dart  <pre>Set<String> names = {'Alice', 'Bob'};</pre>
Queues	FIFO collection for elements	 dart  <pre>import 'dart:collection'; Queue<int> queue = Queue(); queue.addAll([1, 2, 3]);</pre>
LinkedLists	Sequence of elements where each element points to the next	 dart  <pre>import 'dart:collection'; class Item extends LinkedListEntry<Item> { int value; Item(this.value); } LinkedList<Item> list = LinkedList<Item>(); list.add(Item(1));</pre>

Libraries and command line utilities

Type	Description	Code example
import	Access built-in Dart libraries	 dart  <pre>import 'dart:math';</pre>
CLI commands	Compile Dart code to native executable	 dart  <pre>dart compile exe test.dart</pre>
Dart SDK	Essential tool for running and managing Dart applications	 dart  <pre>dart run, dart create</pre>
Pub tool	Package manager to handle dependencies	 dart  <pre>dart pub get, dart pub add http</pre>
Dart DevTools	Suite for debugging and performance profiling	 dart  <pre>Performance profiling, memory analysis, and widget inspection</pre>
dart:core	Fundamental classes and functions	 dart  <pre>Handling strings, numbers, collections like List and Map.</pre>
dart:math	Provides mathematical constants and functions	 dart  <pre>sin, cos, sqrt, and constants like pi.</pre>

dart:async

Supports
asynchronous
programming



dart



Future, Stream **for** handling asynchronous operations.

dart:convert

Handling JSON,
UTF-8
encoding/decoding



dart



jsonEncode, jsonDecode **for** JSON data manipulation.

**Custom
libraries**

Creating and using
your own library



dart



```
library my_utils; int add(int a, int b) => a + b;
```